

PROJETO INTEGRADOR

PLANO DE TESTES

Sistema Soluções Engenharia C&F

Christian Eric Barrantes Briceño

2026

1. Identificação do plano de testes

Item	Descrição
Sistema	Soluções Engenharia C&F
Etapa	Etapa 7 - Planejamento e execução de testes
Desenvolvedor	Christian Eric Barrantes Briceño
Projeto-base	SolucoesEngenhariaCFRefatorado
Tecnologias	Java SE 17, Apache NetBeans, JUnit 4, JDBC, MySQL, Git e GitHub
Repositório	https://github.com/ChrisEricB/SolucoesEngenhariaCFRefatorado
Data de elaboração	2026

Este plano de testes foi elaborado para verificar as principais funcionalidades já implementadas no sistema e orientar os testes das funcionalidades que serão reaproveitadas na futura aplicação web. O documento contempla testes unitários automatizados, testes manuais funcionais, testes de integração e critérios básicos de aceitação.

2. Objetivos

2.1 Objetivo geral

Planejar, executar e registrar testes capazes de demonstrar que as regras de negócio e as principais funcionalidades do sistema Soluções Engenharia C&F apresentam comportamento correto, previsível e compatível com os requisitos definidos para o projeto.

2.2 Objetivos específicos

- Criar e executar testes unitários com JUnit 4 para uma regra de cálculo independente do banco de dados.
- Validar cenários positivos, limites e entradas inválidas da funcionalidade de cálculo de progresso.
- Definir testes manuais para autenticação, usuários, projetos, auditorias e não conformidades.
- Planejar testes de integração com o MySQL e testes futuros para a interface web.
- Registrar evidências da execução dos testes e do versionamento no GitHub.
- Estabelecer critérios de entrada, saída, prioridade e tratamento de defeitos.

3. Escopo dos testes

3.1 Itens incluídos

- Regra de cálculo do percentual de conclusão de um projeto.
- Validações de usuários, projetos, auditorias e não conformidades.
- Autenticação de usuários ativos por e-mail e senha.

- Operações de cadastro, listagem, busca, atualização e exclusão.
- Persistência e consulta de dados no banco MySQL.
- Integração entre as camadas service, repository e repository.jdbc.
- Comportamentos previstos para formulários e navegação da futura aplicação web.
- Versionamento dos testes e das evidências no GitHub.

3.2 Itens fora do escopo desta etapa

- Testes automatizados de interface gráfica ou de navegador.
- Testes de carga, estresse e grande volume de usuários simultâneos.
- Testes de invasão ou auditoria especializada de segurança.
- Automação completa dos testes com integração contínua.
- Testes em ambiente de produção.

4. Estratégia e níveis de teste

Nível	Finalidade	Forma de execução
Teste unitário	Verificar uma regra isolada, sem acesso ao banco.	JUnit 4 no NetBeans.
Teste funcional manual	Validar o comportamento percebido pelo usuário.	Execução de casos de teste com dados válidos e inválidos.
Teste de integração	Verificar comunicação entre serviços, repositórios e MySQL.	Execução do método main() e consultas no banco.
Teste de regressão	Confirmar que alterações não quebraram comportamentos existentes.	Reexecução dos testes unitários e dos casos manuais críticos.
Teste futuro da aplicação web	Validar formulários, mensagens, navegação e persistência.	Execução manual em navegador e posterior automação.

A estratégia prioriza inicialmente regras de negócio independentes do banco, pois são mais simples de isolar e permitem resultados rápidos e repetíveis. As funcionalidades que dependem de persistência serão verificadas principalmente por testes manuais e de integração nesta etapa.

5. Ambiente e pré-requisitos

Item	Configuração prevista
Sistema operacional	Windows 10 ou superior.
Linguagem	Java SE 17.
IDE	Apache NetBeans.
Framework de teste	JUnit 4.

Item	Configuração prevista
Dependência auxiliar	Hamcrest, utilizado internamente pelo JUnit 4.
Banco de dados	MySQL com o schema solucoes_engenharia_cf.
Conector	MySQL Connector/J 9.4.0.
Controle de versão	Git e repositório GitHub.
Dados de teste	Usuários, projetos, auditorias e não conformidades do dump de demonstração.

5.1 Pré-condições gerais

1. O projeto deve abrir corretamente no NetBeans.
2. O JDK 17 deve estar configurado nas Libraries.
3. JUnit 4 deve estar configurado em Test Libraries.
4. O MySQL deve estar iniciado para os testes de integração.
5. O banco solucoes_engenharia_cf deve ter sido criado e importado.
6. As variáveis DB_USUARIO e DB_SENHA devem estar configuradas quando necessárias.
7. Os arquivos do projeto devem estar salvos antes de cada execução.

6. Critérios de entrada e saída

6.1 Critérios de entrada

- Código compilando sem erros de sintaxe.
- Classe ou funcionalidade disponível para execução.
- Ambiente e bibliotecas configurados.
- Dados de teste conhecidos e, quando necessário, banco disponível.
- Caso de teste com resultado esperado definido.

6.2 Critérios de saída

- Todos os testes unitários da classe executados com sucesso.
- Nenhum defeito crítico ou alto aberto nas funcionalidades essenciais.
- Resultados dos testes manuais registrados como aprovado, reprovado ou bloqueado.
- Evidências armazenadas na pasta documentacao/Etapa7.
- Código e testes versionados no GitHub.

7. Testes unitários implementados

7.1 Funcionalidade testada

Foi criada a classe CalculadoraProgressoProjeto, responsável por calcular o percentual de conclusão de um projeto a partir da quantidade de etapas concluídas e do total de etapas. A regra foi separada em uma classe própria para permitir teste unitário sem dependência de interface, banco de dados ou JDBC.

Percentual de conclusão = etapas concluídas × 100 ÷ total de etapas

7.2 Casos de teste unitário

ID	Cenário	Entrada	Resultado esperado
UT01	Calcular progresso parcial.	5 concluídas de 10.	Retornar 50,0%.
UT02	Calcular projeto concluído.	8 concluídas de 8.	Retornar 100,0%.
UT03	Calcular projeto não iniciado.	0 concluídas de 10.	Retornar 0,0%.
UT04	Rejeitar total igual a zero.	0 concluídas de 0.	Lançar IllegalArgumentException.
UT05	Rejeitar concluídas acima do total.	11 concluídas de 10.	Lançar IllegalArgumentException.

7.3 Resultado da execução

Os cinco testes unitários foram executados no NetBeans. A execução apresentou 100% de aprovação, sem falhas e sem erros. Esse resultado demonstra que a regra de cálculo responde corretamente aos cenários válidos, aos valores-limite e às entradas inválidas previstas.

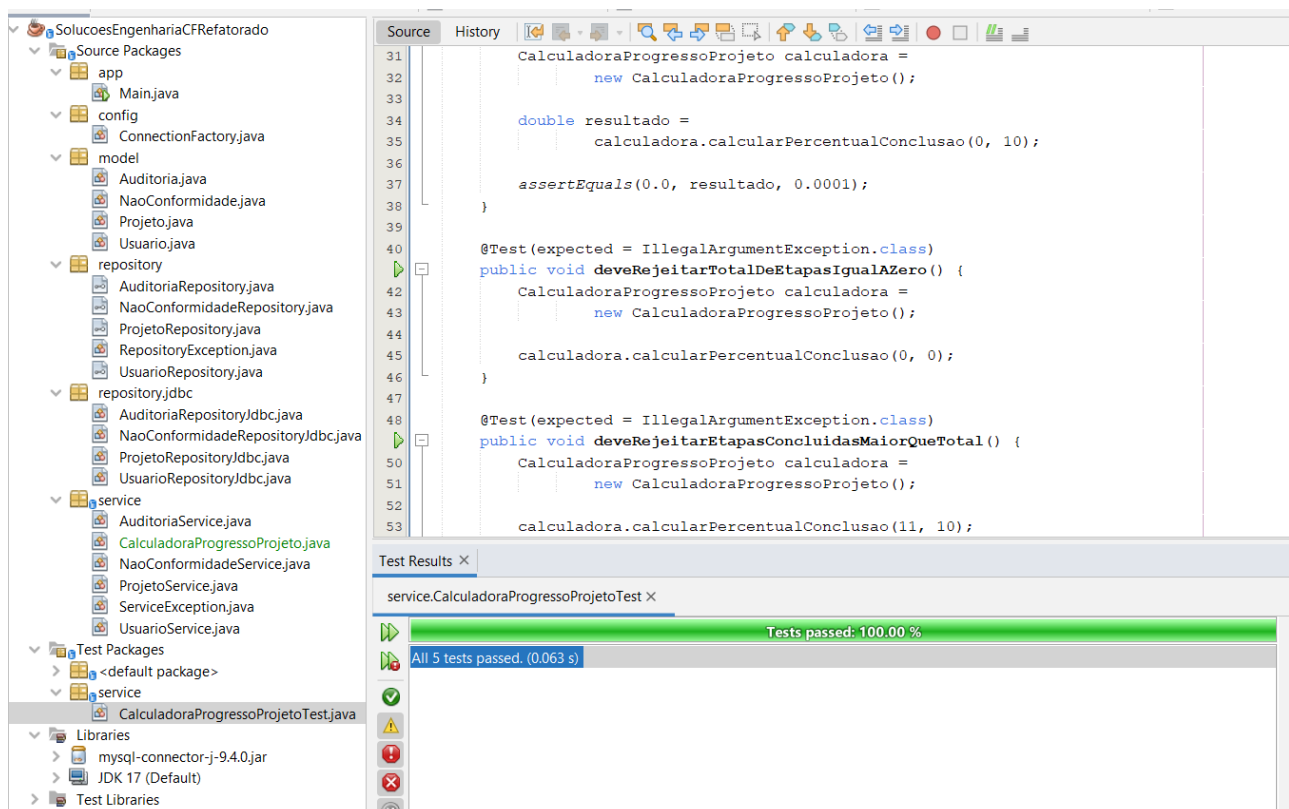


Figura 1 - Execução dos cinco testes unitários com 100% de aprovação no NetBeans.

8. Plano de testes manuais

Os casos abaixo devem ser executados no sistema atual ou adaptados à futura interface web. O campo “Resultado” deverá ser preenchido durante a execução com Aprovado, Reprovado ou Bloqueado.

8.1 Autenticação e usuários

ID	Cenário e procedimento	Resultado esperado	Prioridade
AUT01	Informar e-mail e senha válidos de um usuário ativo.	Autenticação concluída e usuário retornado.	Alta
AUT02	Informar senha incorreta para um e-mail existente.	Sistema informa credenciais inválidas e não autentica.	Alta
AUT03	Tentar autenticar usuário inativo.	Acesso negado.	Alta
USR01	Cadastrar usuário com nome, e-mail, senha e tipo válidos.	Usuário salvo e ID gerado.	Alta
USR02	Cadastrar outro usuário com e-mail já existente.	Cadastro impedido com mensagem de e-mail duplicado.	Alta
USR03	Cadastrar usuário com e-mail em formato inválido.	Validação impede o envio.	Média
USR04	Atualizar dados de um usuário existente.	Dados atualizados sem duplicar o registro.	Média
USR05	Excluir usuário sem vínculos impeditivos.	Usuário removido e não aparece na listagem.	Média

8.2 Projetos e cálculo de progresso

ID	Cenário e procedimento	Resultado esperado	Prioridade
PRO01	Cadastrar projeto com nome, data inicial, status e responsável válidos.	Projeto salvo e apresentado na listagem.	Alta
PRO02	Informar data de término anterior à data de início.	Cadastro ou atualização impedidos.	Alta
PRO03	Informar orçamento negativo.	Validação rejeita o valor.	Alta
PRO04	Cadastrar projeto sem cliente.	Projeto salvo, pois o cliente é opcional.	Média
PRO05	Buscar projeto por ID existente.	Projeto correspondente é retornado.	Média
PRO06	Atualizar status e orçamento de um	Novos dados persistidos.	Média

ID	Cenário e procedimento	Resultado esperado	Prioridade
	projeto.		
PRO07	Excluir projeto sem vínculos impeditivos.	Projeto removido da listagem.	Média
PRO08	Calcular 5 etapas concluídas em um total de 10.	Sistema apresenta 50,0% de progresso.	Média
PRO09	Calcular progresso com total igual a zero.	Sistema rejeita a entrada e informa erro de validação.	Alta

8.3 Auditorias

ID	Cenário e procedimento	Resultado esperado	Prioridade
AUD01	Cadastrar auditoria com projeto, tipo, data e auditor responsável.	Auditoria salva corretamente.	Alta
AUD02	Informar data de realização anterior à data agendada.	Operação rejeitada pela regra de negócio.	Alta
AUD03	Informar resultado sem data de realização.	Operação rejeitada e mensagem apresentada.	Alta
AUD04	Listar auditorias de um projeto específico.	Somente auditorias do projeto são exibidas.	Média
AUD05	Atualizar auditoria existente com data e resultado válidos.	Dados atualizados e persistidos.	Média
AUD06	Excluir auditoria sem vínculos impeditivos.	Registro removido da listagem.	Média

8.4 Não conformidades

ID	Cenário e procedimento	Resultado esperado	Prioridade
NC01	Registrar não conformidade com projeto, descrição, gravidade, responsável e prazo.	Registro salvo com status válido.	Alta
NC02	Informar gravidade diferente de Baixa, Média, Alta ou Crítica.	Operação rejeitada.	Alta
NC03	Definir status Corrigida sem data de correção.	Operação rejeitada e data solicitada.	Alta
NC04	Cadastrar não conformidade sem auditoria vinculada.	Registro salvo, pois a auditoria é opcional.	Média
NC05	Listar não conformidades de um projeto	Somente registros do	Média

ID	Cenário e procedimento	Resultado esperado	Prioridade
	específico.	projeto são exibidos.	
NC06	Atualizar causa raiz, responsável, prazo e status.	Alterações persistidas.	Média
NC07	Excluir não conformidade existente.	Registro removido da listagem.	Média

8.5 Integração e persistência

ID	Cenário e procedimento	Resultado esperado	Prioridade
INT01	Executar o sistema com MySQL iniciado e banco importado.	Conexão realizada e consultas executadas.	Alta
INT02	Executar o sistema com nome de banco incorreto.	Erro de persistência apresentado sem encerrar de forma inesperada.	Média
INT03	Cadastrar um registro e consultar novamente após reiniciar o programa.	Registro permanece disponível no banco.	Alta
INT04	Executar listagens de usuários, projetos, auditorias e não conformidades.	Todos os dados são mapeados sem vazamento de recursos JDBC.	Alta
INT05	Executar operação que viole chave estrangeira.	Falha de persistência informada adequadamente.	Média

8.6 Testes planejados para a aplicação web

ID	Cenário e procedimento	Resultado esperado	Prioridade
WEB01	Abrir a página de login em navegador suportado.	Página carrega sem erros visuais.	Alta
WEB02	Enviar formulário obrigatório com campos vazios.	Mensagens de validação são exibidas junto aos campos.	Alta
WEB03	Autenticar e navegar entre os módulos.	Sessão mantida e páginas autorizadas acessíveis.	Alta
WEB04	Acessar página protegida sem autenticação.	Usuário é redirecionado ao login.	Alta
WEB05	Cadastrar projeto pela interface web.	Dados validados, persistidos e exibidos na listagem.	Alta

ID	Cenário e procedimento	Resultado esperado	Prioridade
WEB06	Editar e excluir registros pela interface.	Operações concluídas após confirmação apropriada.	Média
WEB07	Testar telas em diferentes tamanhos de janela.	Conteúdo permanece legível e utilizável.	Média
WEB08	Atualizar a página após operação concluída.	Estado persistido e mensagens não são duplicadas.	Média

9. Matriz básica de rastreabilidade

Requisito	Descrição resumida	Casos relacionados
RF01	Autenticar usuários ativos.	AUT01, AUT02, AUT03
RF02-RF03	Gerenciar usuários e impedir e-mails duplicados.	USR01 a USR05
RF04-RF05	Gerenciar projetos, responsável e cliente opcional.	PRO01 a PRO07
RF06-RF07	Gerenciar e listar auditorias por projeto.	AUD01 a AUD06
RF08-RF10	Gerenciar não conformidades, prazos e status.	NC01 a NC07
RF11	Persistir dados no banco.	INT01 a INT05
RF12	Informar erros sem ocultar falhas.	AUT02, PRO02, AUD02, NC02, INT02, INT05
Regra nova	Calcular percentual de conclusão do projeto.	UT01 a UT05, PRO08, PRO09
RNF08	Versionar o projeto no GitHub.	Evidência de commit e repositório.
RNF10	Preparar código para aplicação web.	WEB01 a WEB08

10. Registro e classificação de defeitos

Severidade	Definição	Exemplo
Crítica	Impede o uso do sistema ou causa perda/corrupção de dados.	Aplicação não inicia ou banco é corrompido.
Alta	Bloqueia uma funcionalidade essencial sem alternativa viável.	Usuário válido não consegue autenticar.

Severidade	Definição	Exemplo
Média	Afeta uma função, mas existe alternativa temporária.	Filtro por projeto não funciona, mas listagem geral funciona.
Baixa	Problema visual, textual ou de pequena conveniência.	Mensagem com grafia inadequada ou alinhamento incorreto.

Cada defeito deve registrar identificação, data, caso de teste relacionado, ambiente, passos para reprodução, resultado obtido, resultado esperado, evidência e severidade. Após a correção, o caso deve ser reexecutado e incluído no teste de regressão.

11. Controle de execução

Status	Significado
Não executado	O caso ainda não foi iniciado.
Aprovado	O resultado obtido corresponde ao resultado esperado.
Reprovado	O comportamento diverge do esperado e deve gerar defeito.
Bloqueado	Não foi possível executar por dependência, ambiente ou defeito anterior.
Não aplicável	O caso não se aplica à versão avaliada.

11.1 Ordem recomendada de execução

1. Executar Clean and Build no projeto.
2. Executar todos os testes unitários JUnit.
3. Executar os testes manuais de autenticação e usuários.
4. Executar os testes de projetos, auditorias e não conformidades.
5. Executar os testes de integração com o MySQL.
6. Reexecutar os casos críticos após qualquer correção.
7. Registrar resultados e anexar evidências.
8. Versionar o código, os testes e a documentação.

12. Evidências e versionamento

A implementação dos testes foi registrada no repositório GitHub por meio do commit “Adiciona testes unitarios com JUnit”, identificado pelo código d24c077. O commit inclui a classe de cálculo, a classe de testes, a configuração do projeto e a evidência da execução no NetBeans.

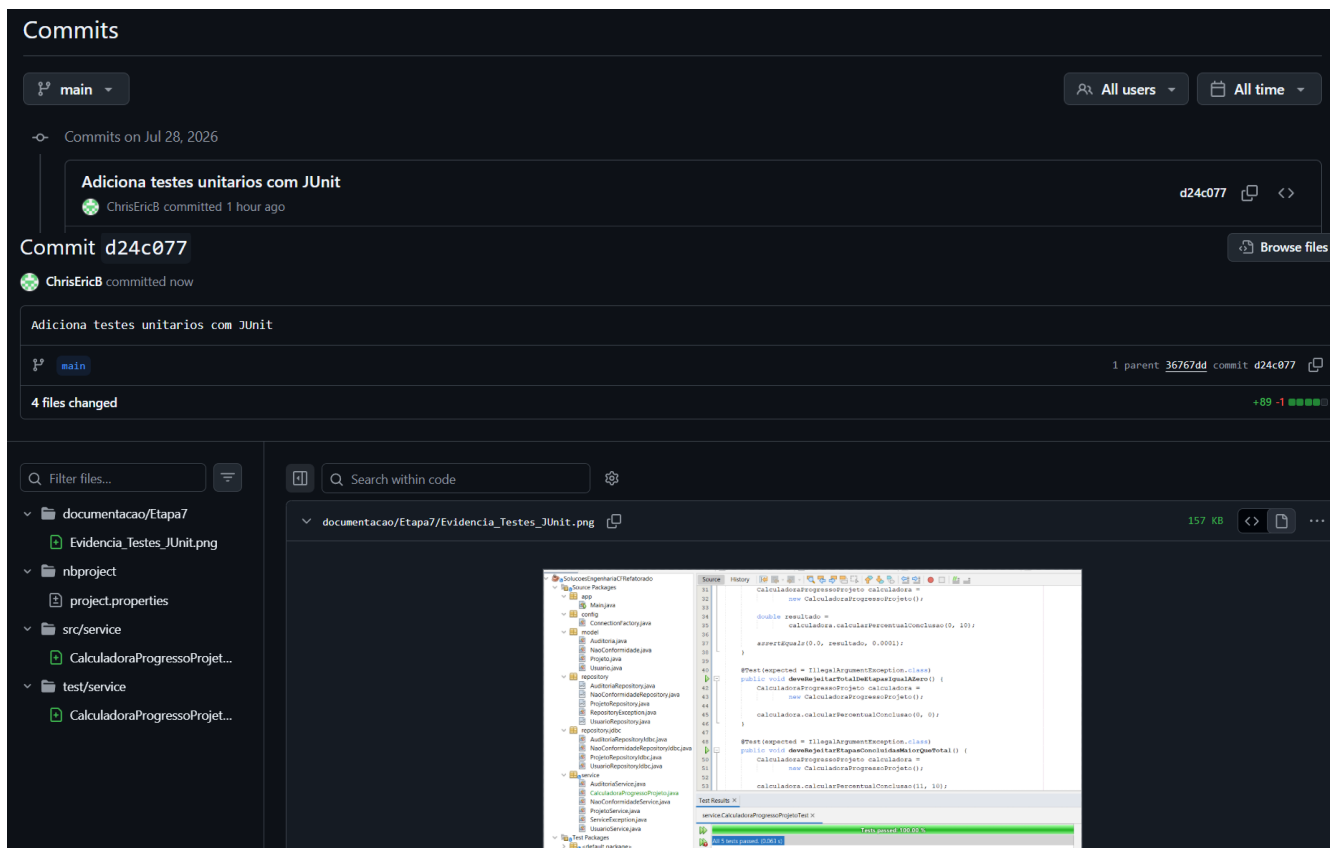


Figura 2 - Evidência do commit que versionou os testes unitários no GitHub.

12.1 Arquivos da Etapa 7

- src/service/CalculadoraProgressoProjeto.java
- test/service/CalculadoraProgressoProjetoTest.java
- documentacao/Etapa7/Evidencia_Testes_JUnit.png
- documentacao/Etapa7/Evidencia_Versionamento_Testes.png
- documentacao/Etapa7/Plano_de_Testes_Etapa7.docx

13. Considerações finais

A Etapa 7 introduziu uma abordagem estruturada de testes no projeto. A criação de uma regra isolada para cálculo de progresso permitiu implementar cinco testes unitários objetivos e independentes do banco de dados. A aprovação de todos os testes demonstra que a funcionalidade trata corretamente os cenários normais, os valores-limite e as entradas inválidas.

O plano manual amplia a cobertura para os principais requisitos existentes e orienta a validação da futura aplicação web. Recomenda-se manter os testes JUnit como parte do processo de regressão e ampliar gradualmente a suíte automatizada à medida que novas regras de negócio forem separadas das camadas de persistência e interface.